

Parnas Tables — Part Deux

Xcode 27 agents design a formal spec and build an app

So, who's still writing code?

Improving Software Quality

Using Formal Methods That Agents Can Build With

Motivation — Correct AI Slop

Using Parnas Tables Review

THE NEW IDEA

Demo Example

Live* Demo!

Summary

Motivation — Correct AI Slop

Motivation — Correct AI Slop

What if the agents output was correct by construction

Fast, and confidently wrong

Agentic coding generates plausible code with subtle bugs

The bottleneck moved

The hard part isn't writing code anymore — it's trusting it

Reviewing code doesn't scale

You can't out-review a firehose of generated diffs — but anyone can check a spec.

Motivation — Correct AI Slop

What if the agents output was correct by construction

Correct by construction, not by luck

Complete + disjoint spec + exhaustive switch

→ the compiler certifies the agent's work

We're not making the agent write more

We're making it impossible for it to be silently wrong

Using Parnas Tables

A brief review

Tabular Notation

Complex decision logic becomes rows × columns a domain expert can read — not nested if/else

Let the Compiler Prove It

Map one row → one switch case. Swift's exhaustive switch enforces completeness at build time

Review the Table Not The Code

The reviewable artifact is the spec. Change the table first; the code follows.

The New Idea

Don't write the table alone
Don't write the app by hand

A Vending App

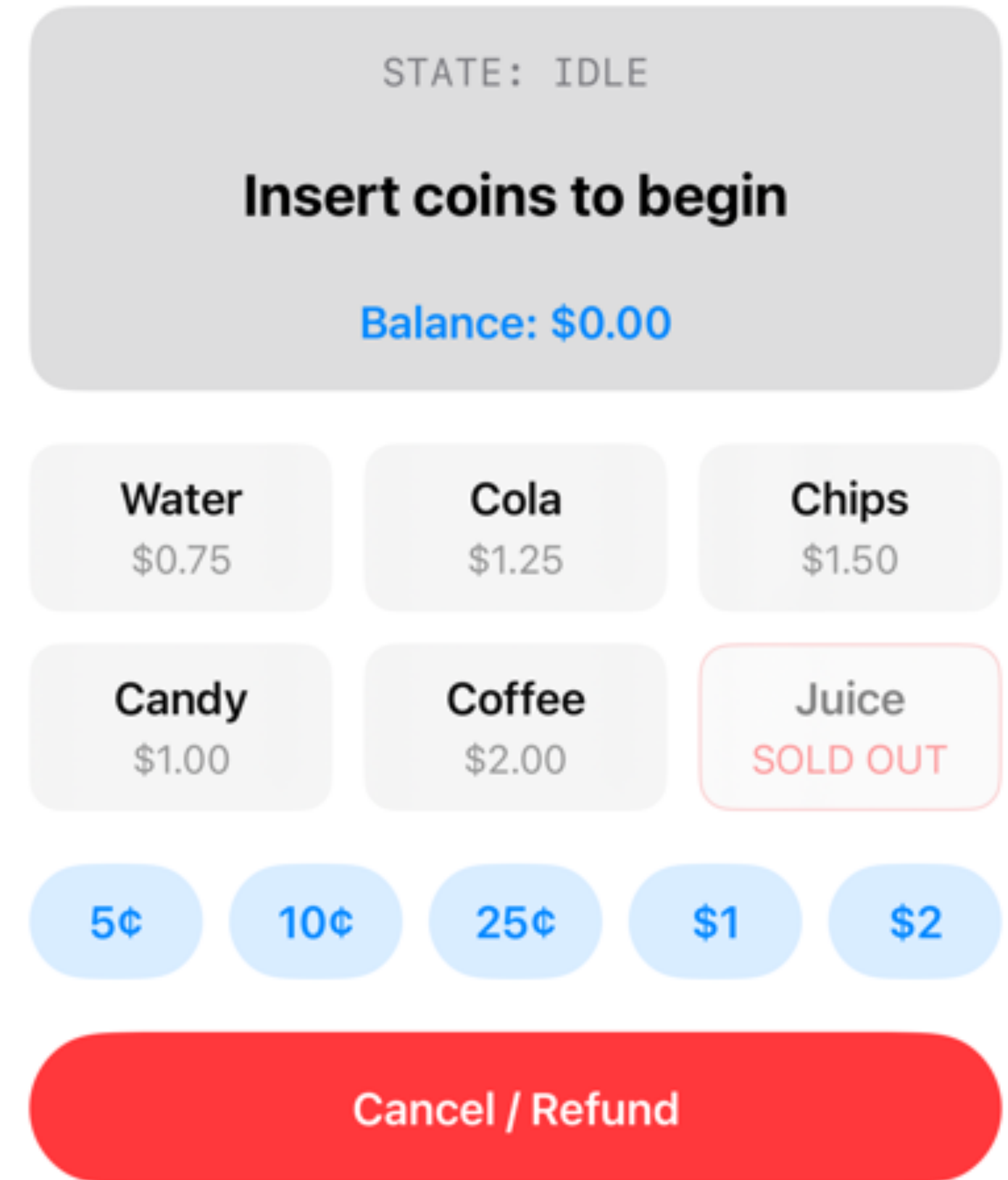
Demo

Builds Fast

3 states, 4 events, one view

Real Guards

“Enough credit?” and “In stock?” — tables define the State × Event × Guard



A Vending App

The Brief From the Domain Expert

- Idle until coins go in. As they do, it collects a balance
- On select: if it's in stock and the balance covers the price, dispense and return change
- If short, say so and keep the money
- If sold out, say so
- Cancel any time for a full refund
- After dispensing, return to idle.

Parnas Table State Machine

Domain Expert Reviewable

#	Current State	Event	Guard Condition	New State	Outputs	Note
1	Idle	InsertCoin(c)	— (assumes $c > 0$)	Collecting	$balance \leftarrow c$	
2	Idle	SelectProduct(p)	—	Idle	"Insert coins first"	no-op; stock not checked (decision (a))
3	Idle	Cancel	—	Idle	—	no-op; idempotent
4	Idle	DispenseComplete	—	Idle	—	impossible ; defensive no-op
5	Collecting	InsertCoin(c)	—	Collecting	$balance \leftarrow balance + c$	
6	Collecting	SelectProduct(p)	$inStock(p) \wedge balance \geq price(p)$	Dispensing	$Dispense(p); ReturnChange(balance - price(p)); balance \leftarrow 0$	money committed here; $ReturnChange(0)$ valid
7	Collecting	SelectProduct(p)	$inStock(p) \wedge balance < price(p)$	Collecting	"Insufficient funds — need $price(p) - balance$ more"	
8	Collecting	SelectProduct(p)	$\neg inStock(p)$	Collecting	"Sold out"	
9	Collecting	Cancel	—	Idle	$Refund(balance); balance \leftarrow 0$	
10	Collecting	DispenseComplete	—	Collecting	—	impossible ; defensive no-op
11	Dispensing	InsertCoin(c)	—	Dispensing	—	no-op; coin slot locked
12	Dispensing	SelectProduct(p)	—	Dispensing	—	no-op; input ignored
13	Dispensing	Cancel	—	Dispensing	—	no-op; cannot cancel mid-vend (nothing to refund)
14	Dispensing	DispenseComplete	—	Idle	"Enjoy"	motor finished; item collected

Rows 6·7·8 share the same State + Event — split only by their guards

Parnas Table State Machine

Completeness: every cell is a decision

	InsertCoin(c)	**SelectProduct(p)**	**Cancel**	**DispenseComplete**
Idle	→ Collecting; balance $\leftarrow c$	<i>no-op</i> ; "Insert coins first"	<i>no-op</i>	<i>impossible</i> ; ignore
Collecting	→ Collecting; balance $+= c$	guarded — 3 rows	→ Idle; Refund	<i>impossible</i> ; ignore
Dispensing	<i>no-op</i> ; slot locked	<i>no-op</i> ; ignored	<i>no-op</i> ; can't cancel	→ Idle; vend done

“Ignore” and “no-op” are choices we state for completeness

Swift's exhaustive switch forces us to

“We forgot” becomes impossible

Parnas Table State Machine

Disjointness: guards partition cleanly

Two independent predicates drive the branch:

- $A = \text{inStock}(p)$
- $B = \text{balance} \geq \text{price}(p)$

The guards written out: (1) $A \wedge B$, (2) $A \wedge \neg B$, (3) $\neg A$. Enumerate all four truth assignments:

A (inStock)	B (affordable)	Row 1 ` $A \wedge B$ `	Row 2 ` $A \wedge \neg B$ `	Row 3 ` $\neg A$ `	Cells matched
T	T	✓	✓		exactly 1
T	F		✓		exactly 1
F	T			✓	exactly 1
F	F			✓	exactly 1

- **No gap:** every one of the 4 rows has exactly one ✓ → the guards *cover* the space.
- **No overlap:** no row has two ✓ → the guards are *mutually exclusive*.

Live Demo

I can't wait to see what we can build right here with Xcode 27

- Agent will plan with a formal spec, write the code, test, and built it
- The plan is a tabular notation artifact a domain expert can review
- Change the spec in the table, the code changes flow through
- Artifacts are visible, traceable, can be committed to version control
- Leverage your favourite model and Apple's agent skills with Xcode 27

Live Demo

The Plan

1. Brief to Parnas Table State Machine
2. Verify Together
3. Generate the Model
4. Generate the App
5. Run it
6. Change the spec, and run again

Live Demo

Summary

How you can start using this today

- Spec first, in a table
 - Make the agent review
 - Let the complier enforce it
 - Change the rule, not the code
-
- Tables in. Verified specs out.
 - Apps that match, by construction 🎉

Summary

The Bigger Picture — A Faster + Cheaper path

- Loop “engineering” is so last week
- Token use is minimized instead of maximized (think of the planet and 💰)
- Flixel Flipside — developed with these methods in 2 weeks
- Xcode 27 agentic coding support supercharges this

Parnas Tables — Part Deux

Xcode 27 agents design a formal spec and build an app

I'm available to coach your team on these methods
mhp@flixel.com

Mark H Pavlidis — tacow — 2026.07.27